

AI Face Detection & Event Photo System

Server Room Deployment Work Order • 30 TB NAS Infrastructure

Company: Lyf Ads
Priority: Production Setup
Date: September 2026

Hi Team,

Thank you for your support! Here are the exact requirements and the 4 tasks needed to set up our AI Face Detection & Event Photo System:

TASK 1: PROVISION A HOST VM (VIRTUAL MACHINE)
• Operating System: Ubuntu 22.04 LTS or 24.04 LTS (or Debian 12) • CPU: 4 to 8 Cores • Memory (RAM): 8 GB to 16 GB RAM • OS Disk: 30 GB to 50 GB SSD • Software Required: Docker Engine & Docker Compose (Can be installed via: <code>curl -fsSL https://get.docker.com sudo sh</code>)
TASK 2: MOUNT THE 30TB NAS PHOTO STORAGE
1. Mount our allocated 30TB NAS partition to the VM at: <code>/mnt/storage/event_photos</code> (Ensure Read & Write permissions: <code>chmod 775</code>) 2. Create an internal SMB / Windows Network Share for this same directory: Path: <code><NAS-IP>\event_photos</code> (e.g., <code>192.168.1.50\event_photos</code>) Purpose: Allows our camera crew to drag-and-drop raw SD card photo folders directly into the storage from their laptops over the office Wi-Fi/LAN.

TASK 3: DEPLOY THE FACE DETECTION APPLICATION (DOCKER)

On the VM, create a directory and run the service using Docker Compose:

```
mkdir -p /opt/face-finder && cd /opt/face-finder
```

Create docker-compose.yml:

```
version: '3.8'

services:
  face-finder:
    image: python:3.11-slim
    container_name: face_finder
    restart: unless-stopped
    ports:
      - "8000:8000"
    volumes:
      # Mount the 30TB NAS folder where event albums are dropped:
      - /mnt/storage/event_photos:/app/storage
      # Persistent vector database for face embeddings:
      - ./qdrant_storage:/app/qdrant_storage
    working_dir: /app
    environment:
      - PORT=8000
      - PYTHONUNBUFFERED=1
    command: >
      bash -c "apt-get update && apt-get install -y git build-essential ffmpeg libsm6 libxext6 &&
        git clone https://github.com/jasirmjr/Face-Detection.git /tmp/repo &&
        cp -r /tmp/repo/* /app/ &&
        pip install --no-cache-dir -r /app/requirements.txt &&
        python /app/run.py"
    deploy:
      resources:
        limits:
          cpus: '8.0'
          memory: 16G
```

Start the service:

```
$ docker compose up -d
```

Verify health check:

```
$ curl http://localhost:8000/health
(Expected output: {"status":"ok","engine_ready":true})
```

TASK 4: PUBLIC HTTPS EXPOSURE VIA CLOUDFLARE TUNNEL

Event guests will search for their photos from mobile phones (4G/5G) at venues:

- Route our subdomain (e.g. photos.ourcompany.com or events.ourcompany.com) through Cloudflare Tunnel to the host at: http://localhost:8000
- This provides free automatic SSL (HTTPS) with zero open firewall ports.

WHAT TO SEND US ONCE COMPLETE:

1. The local network share path for photographers (e.g., \\192.168.1.50\event_photos)
2. The public web URL for guests (e.g., https://photos.ourcompany.com)

